



Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Unidade Curricular de Computação Gráfica

Ano Letivo de 2025/2026

Trabalho Prático — Fase 3 Grupo 31

Afonso Martins

a106931

Francisco Lage

a106813

Gonçalo Castro

a107337

Ricardo Neves

a106850

Abril, 2026

CG

Resumo

Este relatório descreve o trabalho realizado na **3ª Fase** do Trabalho Prático de Computação Gráfica, cujo tema central é a otimização da renderização e a introdução de animações e superfícies curvas.

Foram implementados **Vertex Buffer Objects** (VBOs), substituindo o modo imediato do OpenGL pelo envio único de geometria para a memória da GPU, com reutilização eficiente nos frames seguintes. O formato de ficheiro `.3d` foi mantido e o *engine* passou a suportar também o formato `.obj`.

Introduziram-se **transformações temporais**: rotações com velocidade angular constante e translações ao longo de curvas de **Catmull-Rom**, com suporte opcional ao alinhamento do objeto com a tangente do trajeto a cada instante.

Foi implementada a geração de modelos a partir de **Bezier Patches**, lendo ficheiros `.patch` com pontos de controlo e produzindo malhas trianguladas com nível de detalhe configurável.

A cena de demonstração é um **sistema solar animado**, com planetas, luas e anéis em movimento, e um cometa com trajetória elíptica definida por uma curva de Catmull-Rom e orientação automática ao longo do percurso.

Índice

1. Introdução	3
2. VBOs	3
2.1. Formato dos modelos	3
2.2. Geração de modelos	4
2.3. Renderização através de VBOs	4
3. Transformações Temporais	5
3.1. Simulação de tempo	6
3.2. Rotações	6
3.3. Translações (<i>Catmull-Rom</i>)	7
3.3.1. Interpolação na curva	7
3.3.2. Mapeamento global $\rightarrow$ local	7
3.3.3. Alinhamento à curva	8
4. <i>Bezier Patches</i>	9
4.1. Geração	9
5. Testes	10
5.1. test_3_1.xml — Bule animado em curva de Catmull-Rom	11
5.2. test_3_2.xml — Bule estático em Bezier Patch	12
6. Sistema Solar	13
6.1. Estrutura hierárquica	13
6.2. Corpos celestes	14
6.3. Cometa	15
7. Conclusão	16

1. Introdução

O presente relatório apresenta o trabalho realizado na 3ª Fase do Trabalho Prático da Unidade Curricular Computação Gráfica, pertencente ao 2º Semestre do 3º Ano da Licenciatura em Engenharia Informática, na Universidade do Minho, ano letivo 2025/2026.

Nesta fase foram abordados três tópicos principais:

- **VBOs (*Vertex Buffer Objects*)**: otimização da renderização com envio único de geometria para a GPU, eliminando o tráfego CPU–GPU repetitivo do modo imediato.
- **Transformações temporais**: extensão das rotações e translações para depender do tempo decorrido, permitindo animações — rotações com velocidade angular constante e translações ao longo de curvas de *Catmull-Rom* com alinhamento automático ao trajeto.
- ***Bezier Patches***: nova primitiva geométrica que gera superfícies bicúbicas suaves a partir de grelhas de pontos de controlo, com nível de tecelagem configurável.

Como cena de demonstração, o sistema solar foi inteiramente animado: planetas e luas orbitam com períodos distintos, e um cometa percorre uma trajetória elíptica definida por uma curva de *Catmull-Rom*.

2. VBOs

No modo imediato do OpenGL (fases anteriores), os dados de cada vértice eram enviados individualmente da CPU para a GPU em cada frame, através de chamadas como `glVertex3f`. Esta abordagem gera tráfego constante no bus CPU–GPU e torna-se um gargalo para cenas com muitos vértices.

Os **Vertex Buffer Objects** (VBOs) resolvem este problema: os dados geométricos são transferidos para a memória da GPU uma única vez, sendo reutilizados em todos os frames seguintes sem nova transferência. O resultado é uma renderização significativamente mais eficiente, especialmente para modelos complexos como os gerados por *Bezier Patches*.

2.1. Formato dos modelos

O formato `.3d` foi mantido: a primeira linha contém o número total de vértices e cada linha seguinte define um vértice pelo triplo `x y z`. Os triângulos são formados por grupos consecutivos de três vértices — sem índices explícitos no ficheiro.

```
6
0.5 0.0 0.0
0.0 0.5 0.0
0.0 0.0 0.5
0.5 0.5 0.0
0.5 0.0 0.5
0.0 0.5 0.5
```

Listagem 1: Excerto de ficheiro `.3d` — dois triângulos (6 vértices)

O *engine* passou também a suportar o formato `.obj`, com as diretivas `v x y z` para vértices e `f i1[/t] i2[/t] i3[/t]` para faces indexadas. Os índices são resolvidos na leitura, produzindo a mesma lista plana de vértices usada internamente.

2.2. Geração de modelos

O gerador produz listas planas de vértices no formato `.3d`. Para formas procedurais (esfera, cone, cilindro, etc.), os vértices de cada triângulo são calculados diretamente e escritos em sequência. O novo comando `patch` gera modelos a partir de *Bezier Patches* (Secção 4):

```
generator patch <ficheiro.patch> <tecelagem> <saida.3d>
```

2.3. Renderização através de VBOs

O pipeline de VBOs implementado segue três etapas:

1. **Carregamento (uma vez por modelo):** os vértices são enviados com `glGenBuffers` + `glBufferData(GL_STATIC_DRAW)`. A flag `GL_STATIC_DRAW` indica que os dados não serão modificados, permitindo ao driver GPU alocar na memória mais rápida.
2. **Cache por modelo:** um `std::map<std::string, GLuint>` estático associa o caminho do ficheiro ao ID do buffer na GPU. Se o modelo já foi carregado, o buffer existente é reutilizado — independentemente de quantos grupos o referenciam na cena.
3. **Renderização por frame:** liga-se o buffer, configura-se o atributo com `glVertexPointer(3, GL_FLOAT, 0, 0)` (3 componentes float, stride zero, offset zero) e emite-se `glDrawArrays(GL_TRIANGLES, ...)`.

```
static std::map<std::string, GLuint> vbo_map;

if (vbo_map.find(model.GetName()) == vbo_map.end()) {
    GLuint vbo;
    glGenBuffers(1, &vbo);
    glBindBuffer(GL_ARRAY_BUFFER, vbo);
```

```

glBufferData(GL_ARRAY_BUFFER,
             model.GetVertex().size() * sizeof(Vec3f),
             model.GetVertex().data(), GL_STATIC_DRAW);
vbo_map[model.GetName()] = vbo;
}

glBindBuffer(GL_ARRAY_BUFFER, vbo_map[model.GetName()]);
glEnableClientState(GL_VERTEX_ARRAY);
glVertexPointer(3, GL_FLOAT, 0, 0);
glDrawArrays(GL_TRIANGLES, 0, model.GetVertex().size());
glDisableClientState(GL_VERTEX_ARRAY);
glBindBuffer(GL_ARRAY_BUFFER, 0);

```

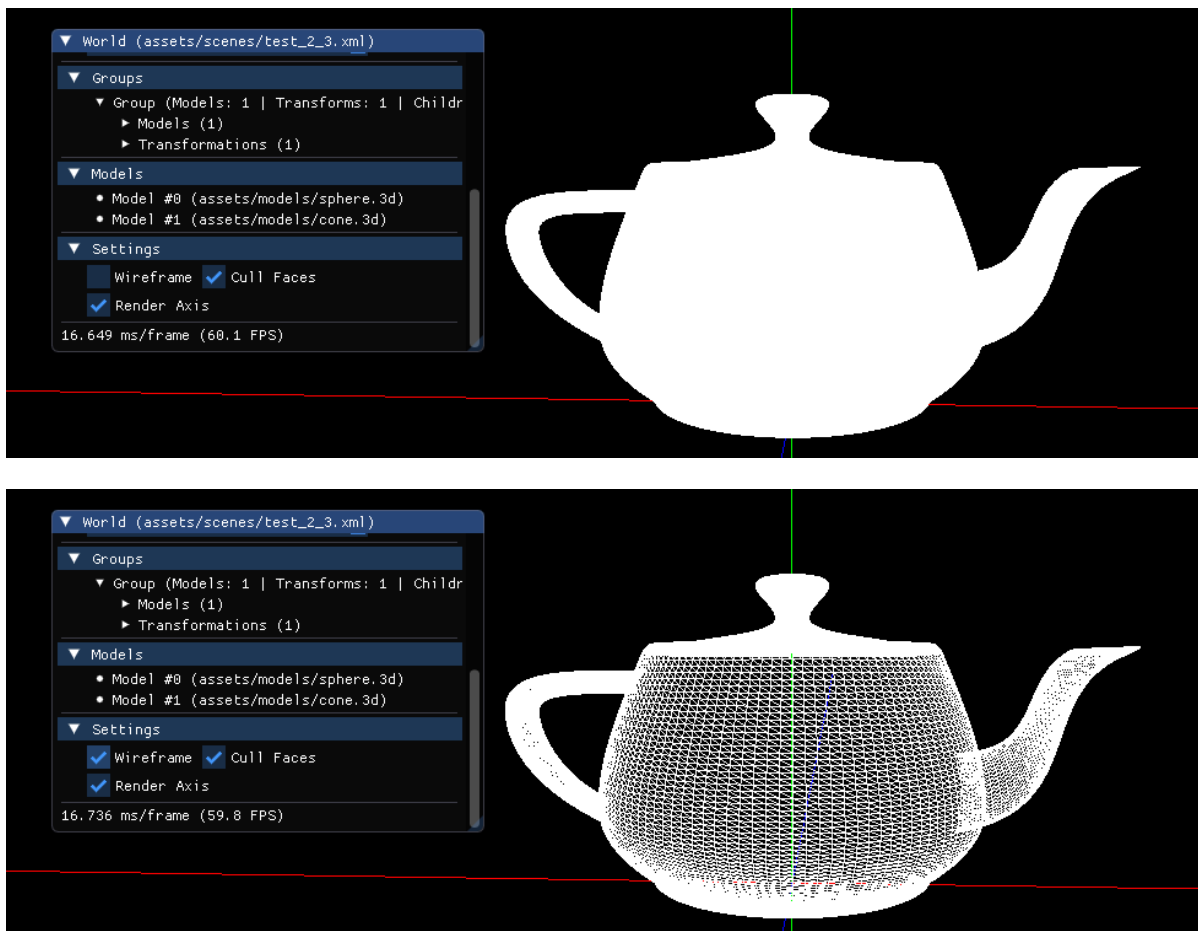


Figura 1: Engine a renderizar via VBOs — modo sólido (cima) e wireframe (baixo) evidenciando a malha de triângulos enviada para a GPU

3. Transformações Temporais

As transformações dos grupos foram estendidas para suportar animações baseadas em tempo. A estrutura `Transform` passou a armazenar um campo `time` (em segundos) que, quando positivo, ativa o modo temporal da transformação. O método `applyTransforms()` do `Group` é chamado a cada frame e consulta o tempo atual.

3.1. Simulação de tempo

O tempo decorrido desde o início da aplicação é obtido através de:

```
float elapsedTime = glfwGetTime();
```

`glfwGetTime()` devolve os segundos em vírgula flutuante desde a inicialização do contexto OpenGL, com precisão sub-milissegundo. Não é necessário acumular deltas manualmente — o valor absoluto é suficiente para calcular ângulos e parâmetros de curva a cada frame.

3.2. Rotações

A rotação temporal faz um grupo girar em torno de um eixo a velocidade constante. O ângulo de rotação no instante t é:

$$\theta(t) = \frac{2\pi \cdot t}{T}$$

onde T é o número de segundos para uma rotação completa de 360° . A transformação aplicada é $R(\theta(t), x, y, z)$. No XML, o atributo `angle` é substituído por `time`:

```
<rotate time="10" x="0" y="1" z="0" />
```

No sistema solar, o Sol roda com `time="25"` e cada planeta tem dois períodos distintos: um para a órbita e outro para a rotação própria (Secção 6).

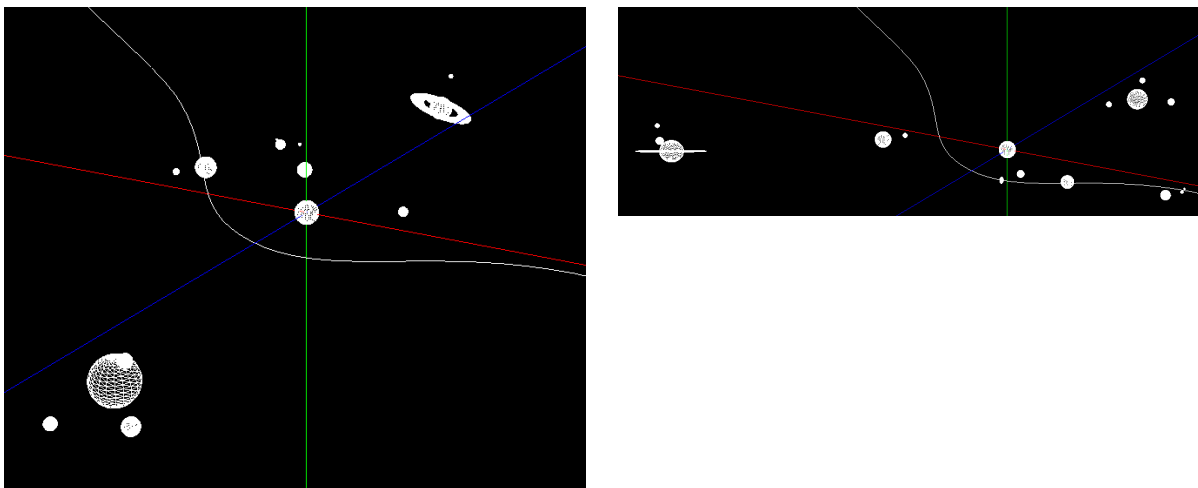


Figura 2: Rotação temporal — planetas em posições orbitais distintas em dois instantes da simulação

3.3. Translações (*Catmull-Rom*)

A translação temporal anima a posição de um grupo ao longo de uma curva de *Catmull-Rom*, percorrendo-a completamente durante a duração configurada.

A curva de *Catmull-Rom* distingue-se por **passar pelos pontos de controlo intermédios** e garantir continuidade C^1 nas junções entre segmentos, sem parâmetros adicionais de tensão. Cada segmento usa os 4 pontos mais próximos.

3.3.1. Interpolação na curva

Utiliza-se a matriz de *Catmull-Rom* $\mathcal{C}$ com tensão $\alpha = 0.5$:

$$\mathcal{C} = \begin{pmatrix} -0.5 & 1.5 & -1.5 & 0.5 \\ 1.0 & -2.5 & 2.0 & -0.5 \\ -0.5 & 0.0 & 0.5 & 0.0 \\ 0.0 & 1.0 & 0.0 & 0.0 \end{pmatrix}$$

Dado o parâmetro local $t \in [0, 1]$ e quatro pontos consecutivos (P_0, P_1, P_2, P_3) , a posição e a derivada são calculadas por componente:

$$\mathcal{P}(t) = \mathcal{C} \cdot \begin{pmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \end{pmatrix} \cdot \begin{pmatrix} t^3 \\ t^2 \\ t \\ 1 \end{pmatrix}$$

$$\mathcal{P}'(t) = \mathcal{C} \cdot \begin{pmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \end{pmatrix} \cdot \begin{pmatrix} 3t^2 \\ 2t \\ 1 \\ 0 \end{pmatrix}$$

3.3.2. Mapeamento global $\rightarrow$ local

O parâmetro global $gt = t_{\text{elapsed}}/T \in [0, 1]$ é mapeado para o segmento e parâmetro local através de indexação modular, garantindo transições suaves e suporte a curvas fechadas:

```
float t      = gt * pointCount;
int  index = (int)floor(t);
t      = t - index;           // parâmetro local em [0, 1)

int p0 = (index + pointCount - 1) % pointCount;
int p1 = index % pointCount;
int p2 = (index + 1) % pointCount;
int p3 = (index + 2) % pointCount;
```

O módulo garante que o último segmento liga ao primeiro ponto de controlo, permitindo trajetos cíclicos. A curva requer **no mínimo 4 pontos de controlo**.

3.3.3. Alinhamento à curva

Quando `align="True"`, o grupo orienta-se a cada instante com a tangente da curva. Constrói-se uma base ortonormal a partir da derivada $\mathcal{P}'(t)$ pelo processo de *Gram-Schmidt*:

$$\vec{X}_i = \mathcal{P}'(t) / \|\mathcal{P}'(t)\| \quad (\text{tangente} \text{ — eixo principal})$$

$$\vec{Z}_i = (\vec{X}_i \times \vec{Y}_{\text{up}}) / \|\vec{X}_i \times \vec{Y}_{\text{up}}\| \quad (\text{binormal})$$

$$\vec{Y}_i = (\vec{Z}_i \times \vec{X}_i) / \|\vec{Z}_i \times \vec{X}_i\| \quad (\text{normal})$$

onde $\vec{Y}_{\text{up}} = (0, 1, 0)$ é o vetor *up* inicial. A matriz de rotação:

$$M = \begin{pmatrix} X_x & Y_x & Z_x & 0 \\ X_y & Y_y & Z_y & 0 \\ X_z & Y_z & Z_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

é aplicada via `glMultMatrixf` após a translação para o ponto calculado:

```
glTranslatef(pos[0], pos[1], pos[2]);
if (transform.align) {
    float X[3] = { deriv[0], deriv[1], deriv[2] };
    normalize(X);
    float Y[3] = { 0, 1, 0 };
    float Z[3];
    crossProduct(X, Y, Z); normalize(Z);
    crossProduct(Z, X, Y); normalize(Y);
    float rotMatrix[16];
    buildRotationMatrix(X, Y, Z, rotMatrix);
    glMultMatrixf(rotMatrix);
}
```

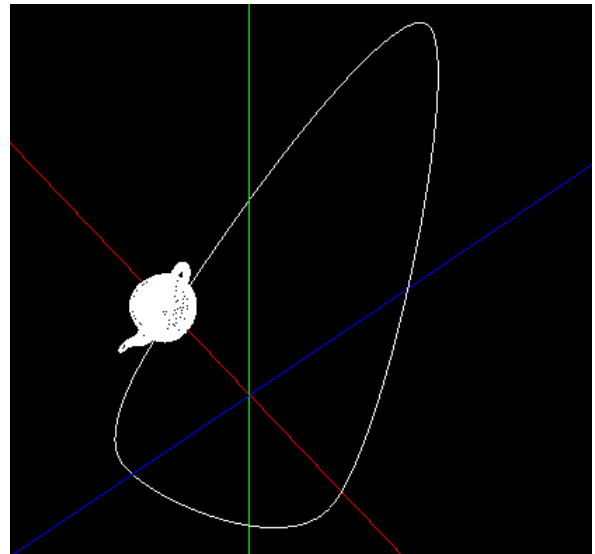
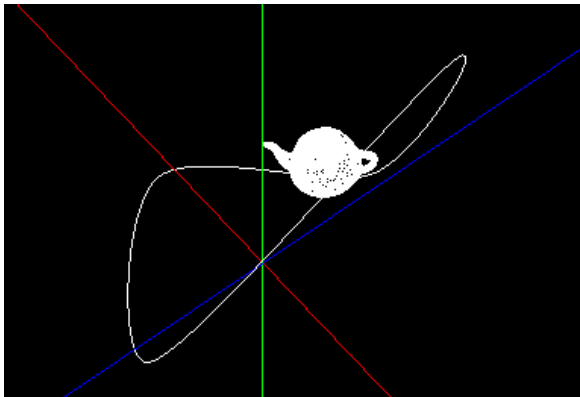


Figura 3: `test_3_1.xml` — bule animado em dois instantes distintos da curva de Catmull-Rom com `align=true`

4. Bezier Patches

Os *Bezier Patches* são superfícies paramétricas bicúbicas definidas por uma grelha 4×4 de pontos de controlo. Qualquer ponto da superfície é determinado pelos parâmetros $(u, v) \in [0, 1]^2$, variando independentemente nas duas direções.

O gerador lê ficheiros `.patch` com o seguinte formato:

```
<nº de patches>
<16 índices separados por vírgulas>      ← um patch por linha
...
<nº de pontos de controlo>
<x>, <y>, <z>                             ← um ponto por linha
...
```

O segundo parâmetro do comando é o nível de **tecelagem**: número de divisões por aresta de cada patch. Um nível 10 produz uma grelha 10×10 de quads (200 triângulos) por patch; um nível 30 produz 1800 triângulos, com muito maior detalhe.

4.1. Geração

Para cada patch, a grelha 4×4 de índices é resolvida para os 16 pontos de controlo P_{ij} . O ponto da superfície para (u, v) é calculado pela equação bicúbica:

$$\mathcal{P}(u, v) = (u^3 \ u^2 \ u \ 1) \cdot M \cdot \begin{pmatrix} P_{00} & P_{01} & P_{02} & P_{03} \\ P_{10} & P_{11} & P_{12} & P_{13} \\ P_{20} & P_{21} & P_{22} & P_{23} \\ P_{30} & P_{31} & P_{32} & P_{33} \end{pmatrix} \cdot M^T \cdot \begin{pmatrix} v^3 \\ v^2 \\ v \\ 1 \end{pmatrix}$$

com a matriz de *Bézier* cúbica $M = M^T$:

$$M = \begin{pmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix}$$

O cálculo é separado por componente (x, y, z) , decomposto em:

1. $UM = (u^3, u^2, u, 1) \cdot M$ — vetor linha de 4 elementos
2. $MV = M \cdot (v^3, v^2, v, 1)^T$ — vetor coluna de 4 elementos
3. $\mathcal{P}_c = \sum_{i,j} (UM)_i \cdot (MV)_j \cdot (P_{ij})_c$

```
float UM[4] = {0,0,0,0}, MV[4] = {0,0,0,0};
for (int i = 0; i < 4; i++)
    for (int j = 0; j < 4; j++) {
```

```

        UM[i] += U[j] * M[j][i];    // U · M
        MV[i] += M[i][j] * V[j];    // M · V
    }
    Vec3f result = {0,0,0};
    for (int i = 0; i < 4; i++)
        for (int j = 0; j < 4; j++)
            result += UM[i] * MV[j] * controlPoints[i*4 + j];

```

Após calculados os pontos da grelha de tecelagem, cada célula (i, j) origina dois triângulos a partir dos seus quatro cantos:

Triângulo	Vértice A	Vértices B e C
1	$P(u_i, v_j)$	$P(u_i, v_{j+1}), P(u_{i+1}, v_j)$
2	$P(u_{i+1}, v_j)$	$P(u_i, v_{j+1}), P(u_{i+1}, v_{j+1})$

Tabela 1: Triângulos gerados por cada célula (i, j) da grelha de tecelagem

```

Vec3f p0 = bezierPatchPoint(u1, v1, patchCPs); // (i, j)
Vec3f p1 = bezierPatchPoint(u2, v1, patchCPs); // (i+1, j)
Vec3f p2 = bezierPatchPoint(u1, v2, patchCPs); // (i, j+1)
Vec3f p3 = bezierPatchPoint(u2, v2, patchCPs); // (i+1, j+1)

vertices.push_back(p0); vertices.push_back(p2); vertices.push_back(p1);
vertices.push_back(p1); vertices.push_back(p2); vertices.push_back(p3);

```

A ordem dos vértices garante que as normais (face exterior) estejam orientadas de forma consistente em todos os triângulos.

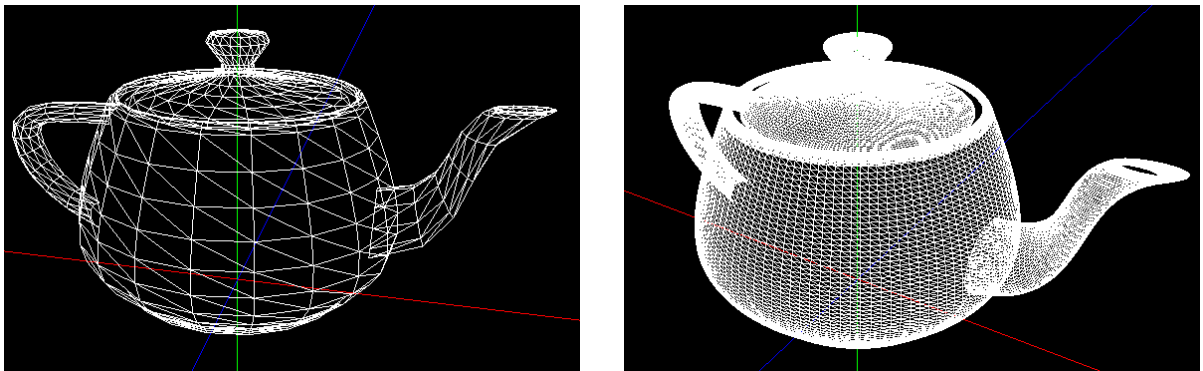


Figura 4: Comparação de níveis de tecelagem no modelo do bule — esquerda: tecelagem 5, direita: tecelagem 30

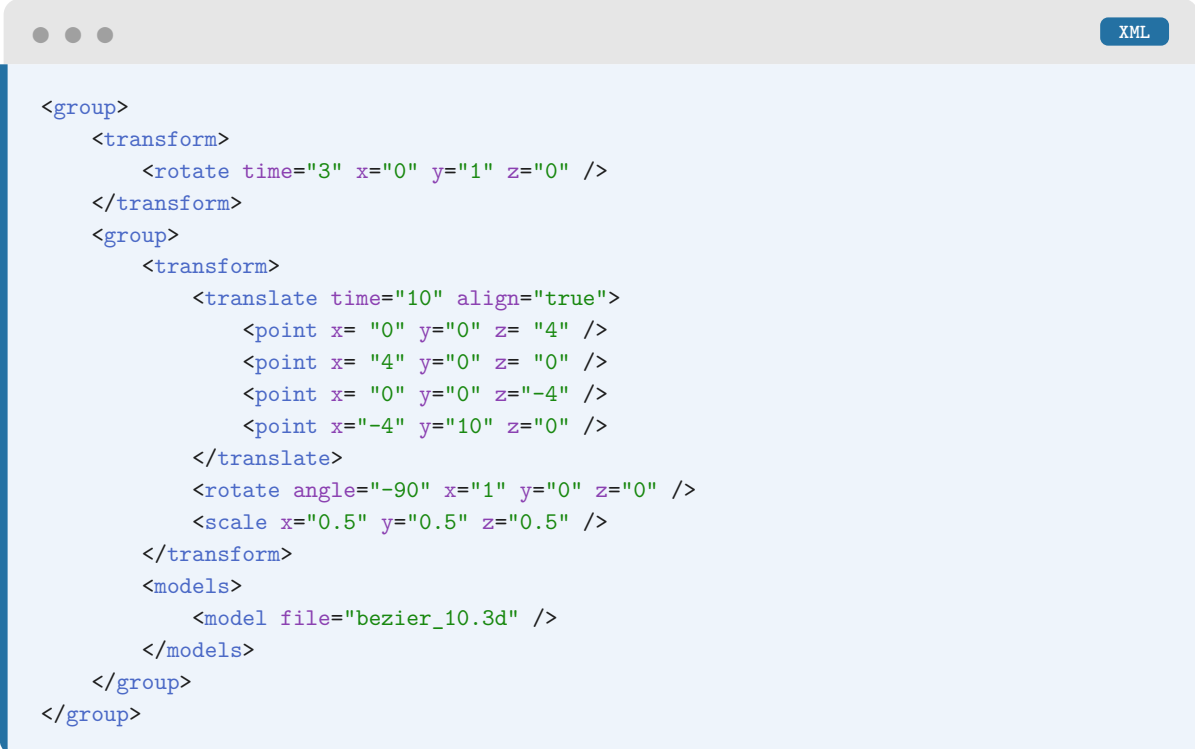
5. Testes

Para validar as implementações desta fase foram criadas duas cenas de teste específicas, disponíveis em `assets/test_files/test_files_phase_3/`.

5.1. test_3_1.xml — Bule animado em curva de Catmull-Rom

Este teste valida em simultâneo: *Bezier Patches*, translação temporal com *Catmull-Rom* e alinhamento à curva.

O bule (`bezier_10.3d`) é gerado com tecelagem 10 a partir do ficheiro `teapot.patch` e animado ao longo de uma curva fechada com 4 pontos de controlo, percorrida em 10 segundos. O atributo `align="true"` ativa o alinhamento:



```
<group>
  <transform>
    <rotate time="3" x="0" y="1" z="0" />
  </transform>
  <group>
    <transform>
      <translate time="10" align="true">
        <point x= "0" y="0" z= "4" />
        <point x= "4" y="0" z= "0" />
        <point x= "0" y="0" z="-4" />
        <point x="-4" y="10" z="0" />
      </translate>
      <rotate angle="-90" x="1" y="0" z="0" />
      <scale x="0.5" y="0.5" z="0.5" />
    </transform>
    <models>
      <model file="bezier_10.3d" />
    </models>
  </group>
</group>
```

Nota: a rotação estática de -90° em X corrige a orientação original do bule (que por padrão fica deitado). A escala de 0.5 ajusta o tamanho para a cena. O grupo externo com `rotate time="3"` faz o conjunto girar lentamente, evidenciando a tridimensionalidade da curva.

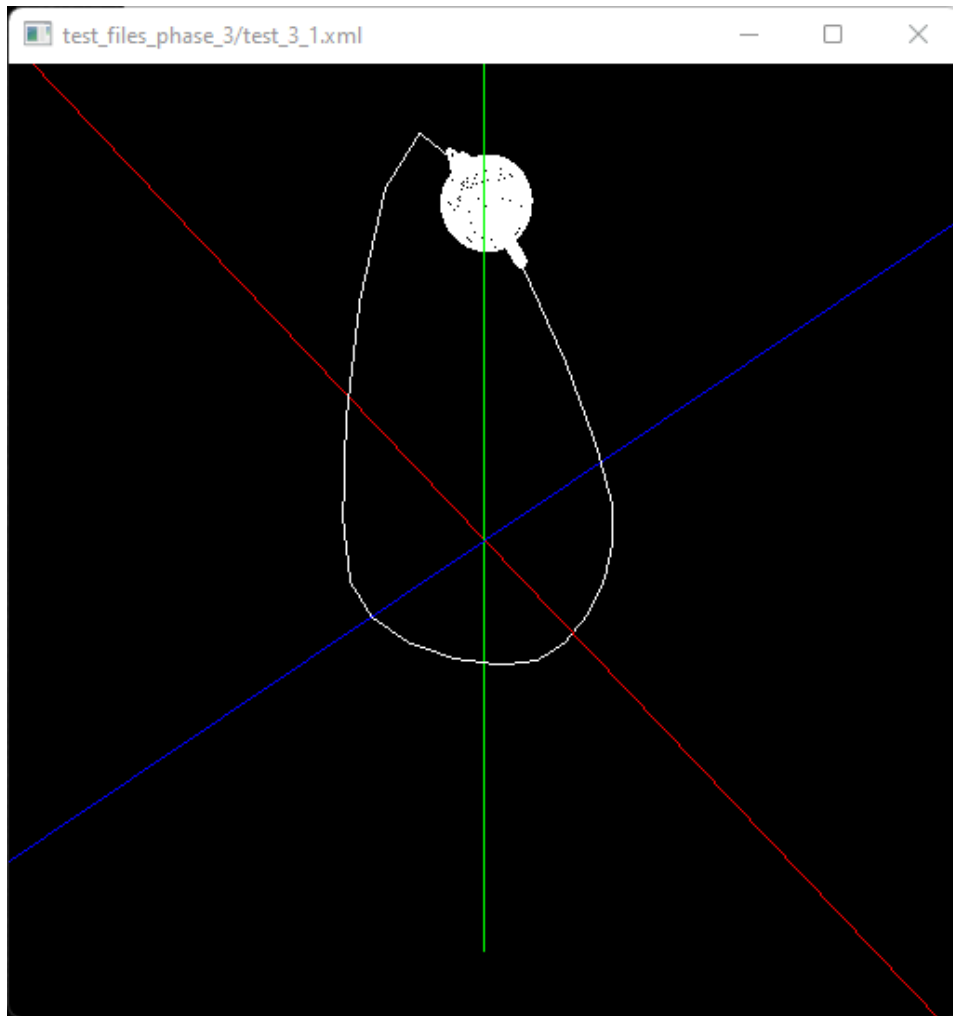


Figura 5: test_3_1.xml — bule animado em curva de Catmull-Rom com alinhamento

5.2. test_3_2.xml — Bule estático em Bezier Patch

Este teste mais simples valida apenas a geração e renderização via VBO do modelo de *Bezier Patch*, sem animações. O bule é apresentado com uma rotação estática de -90° em X para ficar na orientação correta:

```

<group>
  <transform>
    <rotate angle="-90" x="1" y="0" z="0" />
  </transform>
  <models>
    <model file="bezier_10.3d" />
  </models>
</group>

```

Este teste permite verificar a correção geométrica do modelo gerado — todas as faces devem estar orientadas corretamente e a superfície deve ser suave e contínua entre patches adjacentes.

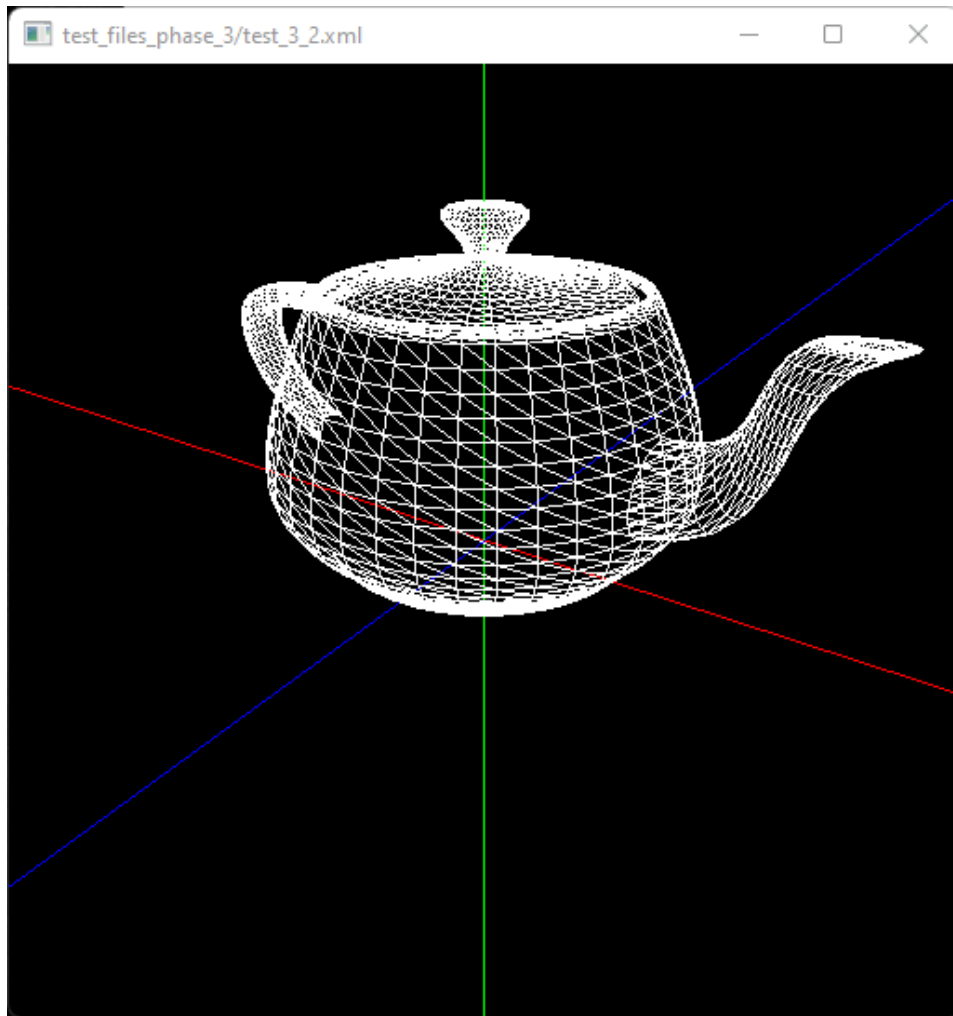


Figura 6: test_3_2.xml — bule estático gerado por Bezier Patch, renderizado via VBO

6. Sistema Solar

O sistema solar foi inteiramente animado nesta fase. Cada corpo celeste possui **dois níveis de rotação temporal**: uma orbital (em torno do Sol) e uma própria (auto-rotação). A hierarquia de grupos no XML compõe estas animações naturalmente.

6.1. Estrutura hierárquica

O padrão utilizado para cada planeta:

```
<group>
  <transform>
    <!-- rotação orbital: define o período de órbita -->
    <rotate time="T_orbit" x="0" y="1" z="0" />
  </transform>
  <transform>
    <!-- translação estática: define a distância ao Sol -->
```

```

    <translate x="dist" y="0" z="0" />
  </transform>
  <group>
    <!-- auto-rotação e escala do planeta -->
    <transform>
      <rotate time="T_self" x="0" y="1" z="0" />
      <scale x="s" y="s" z="s" />
    </transform>
    <models> <model file="assets/models/sphere.3d" /> </models>
  </group>
</group>

```

A rotação orbital é aplicada **antes** da translação: o planeta orbita o Sol à distância correta. As luas são grupos irmãos do planeta, herdando a rotação orbital mas com a sua própria translação e rotação.

6.2. Corpos celestes

Corpo	T orbital (s)	Distância	T próprio (s)	Escala
Sol	—	—	25	1.0
Mercúrio	4	8	2	0.4
Vénus	7	12	3	0.7
Terra	10	16	2	1.0
Lua	1.5	3*	1.5	0.3
Marte	18	21	2	0.5
Júpiter	25	30	1	1.5
Saturno	35	40	1.5	1.3

Tabela 2: Parâmetros de animação dos corpos celestes (* distância relativa ao planeta)

Saturno inclui um anel modelado com um toro achatado (`scale x=2, y=0.2, z=2`) com inclinação estática de 20° no eixo X. Júpiter e Marte têm múltiplas luas com eixos de rotação orbital distintos para maior variedade visual.

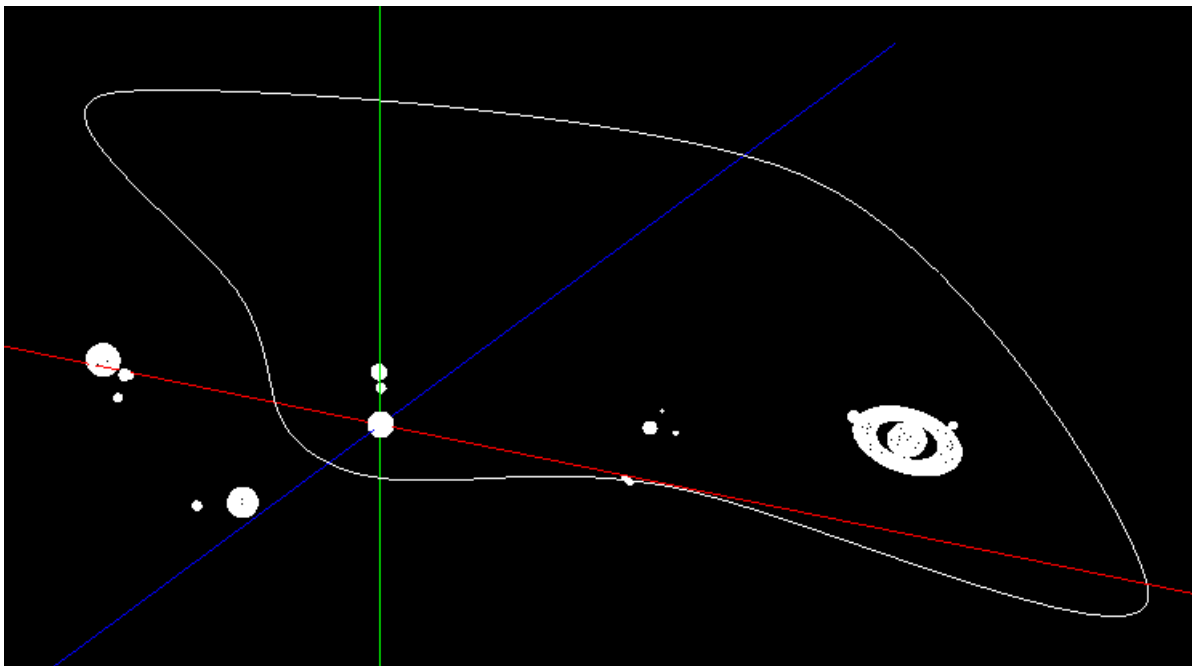


Figura 7: Sistema solar animado — vista geral com todos os corpos celestes

6.3. Cometa

O cometa segue uma trajetória de *Catmull-Rom* com 6 pontos de controle, descrevendo uma órbita excêntrica inclinada em torno do Sol, percorrida em 15 segundos. O ponto $(0, 0, 8)$ representa o periélio — a posição de maior aproximação ao Sol.

```

<translate time="15" align="true">
  <point x="-60" y="20" z="-40" />
  <point x="-20" y="5" z="-10" />
  <point x="0" y="0" z="8" /> <!-- periélio -->
  <point x="20" y="-5" z="-10" />
  <point x="60" y="-20" z="-40" />
  <point x="20" y="10" z="-80" />
</translate>

```

O parâmetro `align="true"` orienta o cometa com a tangente da curva. Uma escala assimétrica $(0.2, 0.2, 0.8)$ confere uma forma alongada no eixo do movimento, evocando a aparência característica de um cometa.

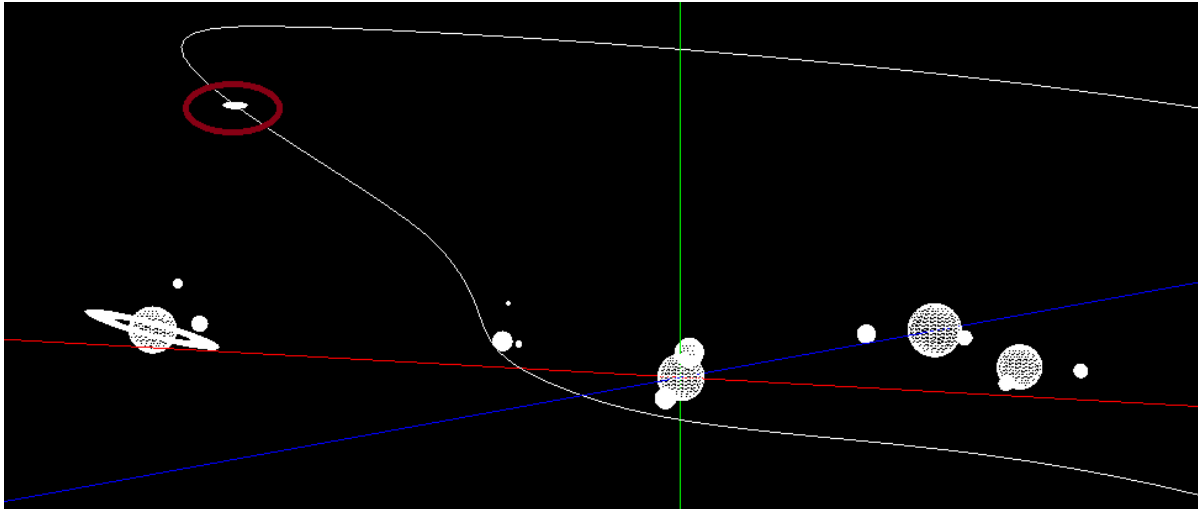


Figura 8: Cometa a percorrer trajetória de Catmull-Rom com `align=true`

7. Conclusão

Nesta fase foram implementadas com sucesso todas as funcionalidades propostas:

- **VBOs:** a renderização foi otimizada com envio único de geometria para a GPU, usando um cache por nome de modelo. A flag `GL_STATIC_DRAW` garante alocação otimizada na memória da GPU.
- **Transformações temporais:** rotações e translações baseadas no tempo de `glfwGetTime()` permitem animações fluidas. A translação *Catmull-Rom* garante continuidade C^1 e suporta alinhamento automático via base de *Gram-Schmidt*.
- **Bezier Patches:** a equação bicúbica parametrizada em (u, v) gera superfícies suaves a partir de ficheiros `.patch`, com nível de detalhe controlado pelo parâmetro de tecelagem. Os dois testes (`test_3_1.xml` e `test_3_2.xml`) validam a correção geométrica e a integração com as transformações temporais.

O sistema solar demonstra todas as funcionalidades em conjunto. O grupo encontra-se em boa posição para a implementação da 4ª Fase, que incluirá iluminação e texturas.